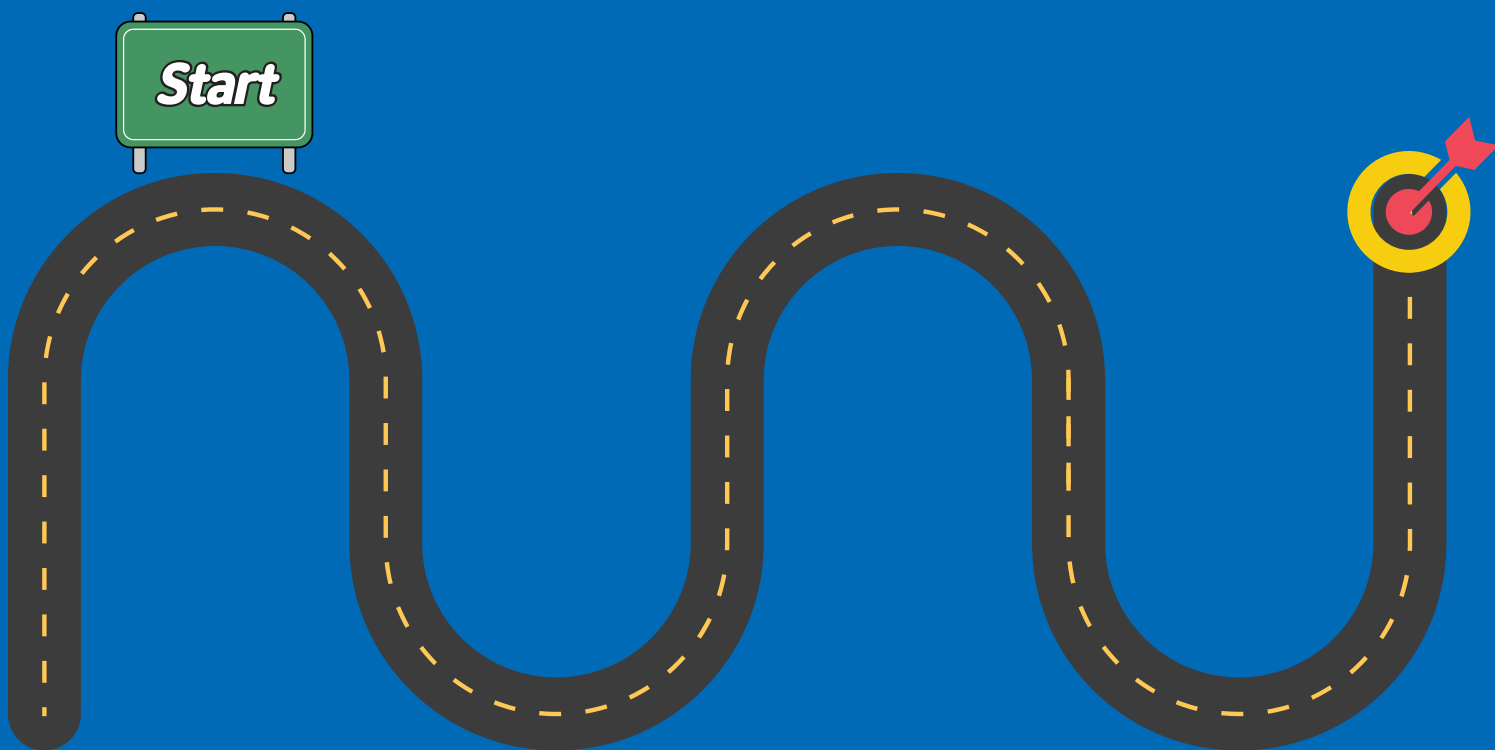


스프링프레임워크

1부: 기본 기능 구성



학습목표

챕터별 북마크의 프로젝트의 기능을 구현하자

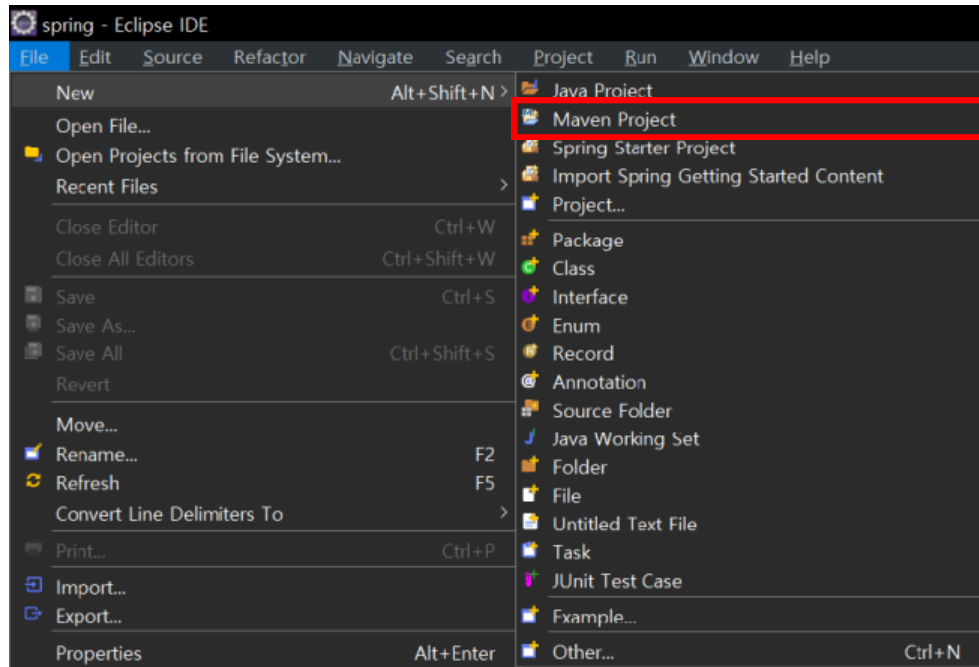
1. 스프링 프로젝트 생성 및 초기화

학습 목표 : 스프링 프레임워크 프로젝트를 생성하고 구성요소를 작성할 수 있다.

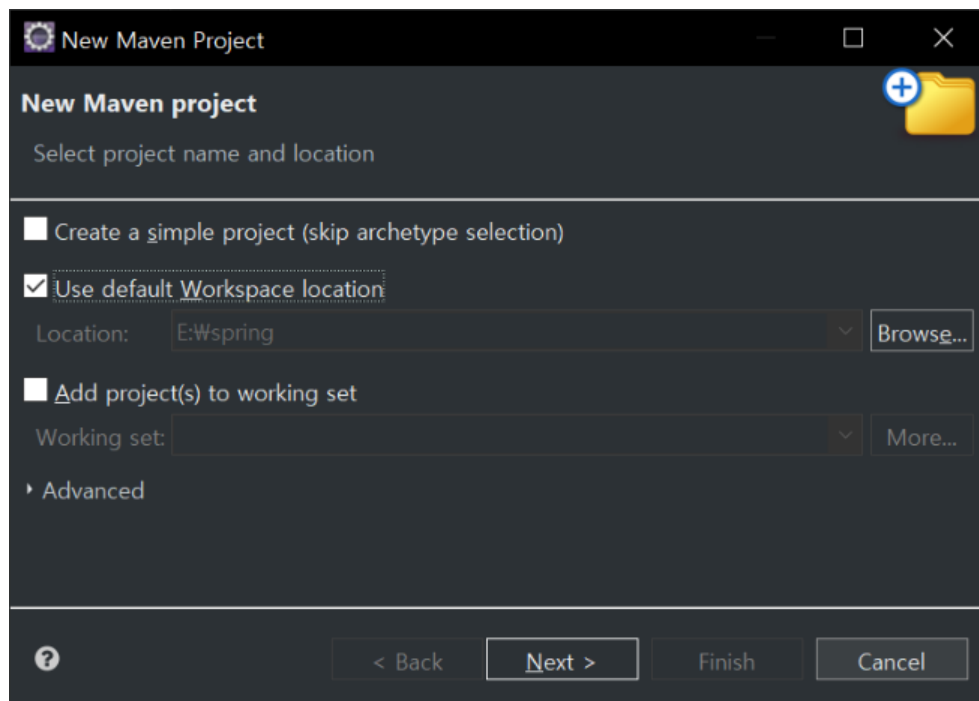


스텝 1:

프로젝트 생성하기



Maven Project를 선택하여 새로운 프로젝트를 생성합니다.



워크스페이스를 적절하게 선택합니다.

New Maven Project

Select an Archetype

Catalog: Internal

Filter: webapp

Group Id	Artifact Id	Version
org.apache.maven.archetypes	maven-archetype-webapp	1.0

☒ Show the last version of Archetype only ☐ Include snapshot archetypes Add...

▸ Advanced

? < Back Next > Finish Cancel

Filter에 webapp을 선택하여 Archetype을 선택합니다.

New Maven Project

Specify Archetype parameters

Group Id: com.springmvc

Artifact Id: BookStore

Version: 0.0.1-SNAPSHOT

Package: com.springmvc.BookStore

☒ run archetype generation interactively

Properties available from archetype:

Name	Value

Add... Remove

? < Back Next > Finish Cancel

Group Id : groupId는 프로젝트의 그룹 또는 조직을 식별하는 고유한 ID입니다. 일반적으로 도메인 이름을 거꾸로 쓴 형태로 사용됩니다. 예를 들어, com.example와 같은 형식입니다. 이 값은 프로젝트가 속한 그룹을 나타내며, 여러 프로젝트가 같은 그룹에 속할 수 있습니다

Artifact Id : artifactId는 특정 프로젝트 또는 모듈을 식별하는 고유한 ID입니다. 이 값은 해당 프로젝트의 이름을 나타내며, 같은 그룹 내에서 유일해야 합니다.

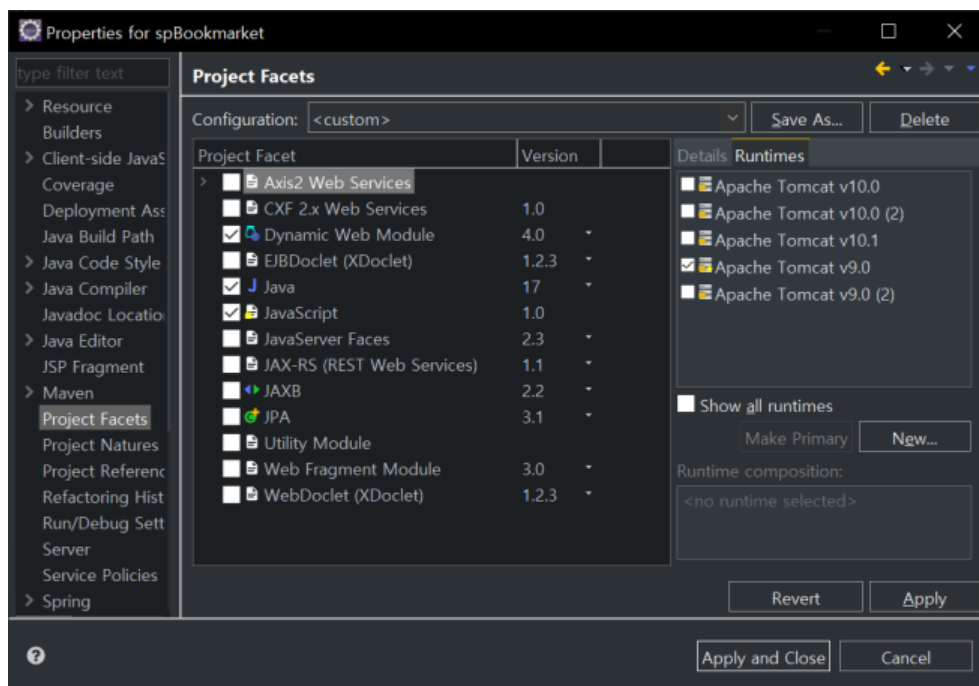
```
Problems Javadoc Declaration Progress CodeTogether Live Servers Console x
<terminated> C:\Program Files\Java\jdk-17\bin\javaw.exe (2024. 11. 29. 오후 2:58:53) [pid: 17500]
package: com.springmvc.BookStore
Y:
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: mave
[INFO] -----
[INFO] Parameter: basedir, Value: E:\spring
[INFO] Parameter: package, Value: com.springmvc.BookStore
[INFO] Parameter: groupId, Value: com.springmvc
[INFO] Parameter: artifactId, Value: BookStore
[INFO] Parameter: packageName, Value: com.springmvc.BookStore
[INFO] Parameter: version, Value: 0.0.1-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: E:\spring\BookStore
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.684 s
[INFO] Finished at: 2024-11-29T14:59:06+09:00
[INFO] -----
```

프로젝트 생성을 실행하면 다음과 같이 콘솔에 출력되며 Y: 부분에서 Enter를 입력해야 빌드가 성공한다.

 스텝 2:

프로젝트 설정 수정하기

생성된 프로젝트의 이름에서 마우스오른쪽을 클릭하여 Property항목을 클릭하여 다음의 화면을 연다



생성된 프로젝트의 이름에서 마우스오른쪽을 클릭하여 Property항목을 클릭하여 다음의 화면을 연다
다음의 항목을 아래와 같이 수정합니다.

- Dynamic Web Module : 4.0
- Java : 17
- Apache Tomcat v9.0

<project> 태그안에 3개의 태그를 생성하거나 수정합니다.

- <properties> key, value 사용
- <dependencies>
- <build>

<properties> 태그를 새로 생성한다. : 특정값을 변수처리하여 호출 사용할 수 있다.

```
<properties>
  <java-version>17</java-version>
  <org.springframework-version>5.3.19</org.springframework-version>
  <org.aspectj-version>1.9.9.1</org.aspectj-version>
  <org.slf4j-version>1.7.36</org.slf4j-version>
</properties>
```

<dependencies> 태그안에 다음의 태그를 삽입한다.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>${org.springframework-version}</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>${org.springframework-version}</version>
</dependency>
```

<build> 태그안에 다음의 태그를 삽입한다.

```
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>2.5.1</version>
    <configuration>
      <source>17</source>
      <target>17</target>
      <compilerArgument>-Xlint:all</compilerArgument>
      <showWarnings>true</showWarnings>
      <showDeprecation>true</showDeprecation>
    </configuration>
  </plugin>
  <plugin>
    <artifactId>maven-war-plugin</artifactId>
    <version>3.2.2</version>
  </plugin>
</plugins>
```

 스텝 4 :

web.xml 작성하기

<webapp> 태그안에 객체 생성을 위한 태그를 작성합니다.

- Root-Context 객체 저장소 만들기.
- Servlet-Context 객체 저장소 만들기.

Root-Context : <listener> 태그와 <context-param>태그

- web.xml이 로드될때 자동으로 실행되는 태그이다.
- ContextLoaderListener는 Spring의 애플리케이션 컨텍스트(공유빈관리)를 초기화한다.
- 웹 애플리케이션의 생명 주기 동안 관리하는 역할을 한다.
- 리스너는 contextConfigLocation에서 지정한 설정 파일(xml)을 읽어 스프링 컨테이너를 생성.
- 리스너는 개발자가 명시적으로 호출하지 않아도 웹 어플리케이션(프로젝트)의 생명 주기와 함께 자동으로 실행된다.

<listener> 태그와 <context-param> 태그

```
<listener>
  <listener-class>
    org.springframework.web.context.ContextLoaderListener
  </listener-class>
</listener>

<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>
    /WEB-INF/spring/root-context.xml
  </param-value>
</context-param>
```

스프링컨테이너	
루트컨테이너	서블릿컨테이너
전역 컨텍스트 -공유-	루트의 자식 컨텍스트 -독립적-

Servlet-Context : <servlet> 태그와 <servlet-mapping>태그

- web.xml이 로드 될때 DispatcherServlet객체(저장소)를 생성한다.
- 생성된 Servlet-Context저장소의 객체는 접속되는 유저마다 개별적으로 제공된다.
- servlet-context.xml에 지정된 객체를 생성하고 제공한다.
- 서블릿 매핑에 의해 모든 요청은 DispatcherServlet객체로 전달된다.
- 전달된 요청을 서블릿컨텍스트에 지정된 객체(핸들러매핑,핸들러어댑터,뷰리졸브)에 의해 처리된다

<servlet> 태그로 DispatcherServlet 객체를 생성한다.

```
<servlet>
  <servlet-name>appServlet</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/spring/appServlet/servlet-context.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>appServlet</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```



스텝 5:

root-context.xml 작성하기

root-Context : 전역 빈(객체)을 설정합니다.

- 전체 유저가 공유(공동)하여 사용할 빈을 설정합니다.
- 주로 비즈니스로직, 데이터 액세스 계층, 서비스 계층등과 관련된 빈을 주로 생성합니다.
- web.xml에 지정된 /WEB-INF/spring/root-context.xml 위치에 파일 생성.
- 전달된 요청을 서블릿컨텍스트에 지정된 객체(핸들러매핑,핸들러어댑터,뷰리졸브)에 의해 처리된다

<beans> 태그 안에 생성할 객체를 정의한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans https://www.springframework.org/schema/beans/spring-beans.xsd">
  <!-- Root Context: 모든 유저가 공유하여 사용하는 빈을 설정합니다. -->
  <!-- 스캔하여 객체 생성하기 -->
  <context:component-scan base-package="com.example.*" />
  <!-- 수동으로 객체 생성하기 -->
  <bean id="datasource" class="org.apache.commons.dbch.BasicDataSource">
    <preproperty name="driverClassName" value="com.mysql.cj.jdbc.Driver" />
    <preproperty name="url" value="jdbc:mysql://localhost:3306/mydb" />
    <preproperty name="username" value="root" />
    <preproperty name="password" value="password" />
  </bean>
</beans>
```



스텝 6 :

servlet-context.xml

servlet-context : 독립적인 빈을 저장하는 공간을 생성합니다.

- 서블릿 컨텍스트 객체를 통하여 직접연결되어 실행되는 객체를 주로 생성합니다.
- Root-Context의 객체를 직접적으로 사용할 수 있으나 다른 유저의 서블릿 컨텍스트는 접근불가
- 주로 컨트롤러와 뷰에 관련된 객체를 주로 생성하고 관리한다.

<beans:bean> 태그의 속성과 스키마를 선언하고 태그안에 객체를 생성합니다.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans:beans
  xmlns="http://www.springframework.org/schema/mvc"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/mvc/spring-mvc.xsd
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd">
  <!-- 생성할 빈을 선언하는곳 -->
</beans:beans>
```

<beans:bean> 태그 안에 뷰와 관련된 빈 생성하기

```
<?xml version="1.0" encoding="UTF-8"?>
<beans:beans ... 스키마 생략>
  <!-- 뷰 리소스 관련 경로 지정 : Servlet-context에서 핸들러 매핑을 거치지 않음 -->
  <resources mapping="/resources/**" location="/resources/" />
  <!-- 뷰 리졸버 -->
  <beans:bean
    class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <beans:property name="prefix" value="/WEB-INF/views/" />
    <beans:property name="suffix" value=".jsp" />
  </beans:bean>
</beans:beans>
```

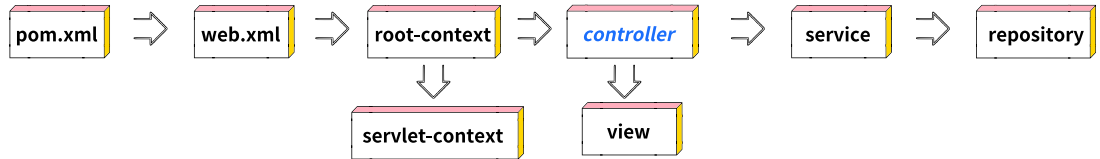
<beans:bean> 태그 안에 컨트롤러와 관련된 빈 생성하기

```
<?xml version="1.0" encoding="UTF-8"?>
<beans:beans ... 스키마 생략>
  <!-- 핸들러 매핑과 아답터 빈 생성 -->
  <annotation-driven />
  <!-- 컨트롤러 수동 빈 생성 방법 -->
  <bean id="controller" class="com.springmvc.controller.Mycontroller />
  <!-- 컨트롤러 자동 빈 생성 방법 -->
  <context:component-scan base-package="com.springmvc.controller" />
</beans:beans>
```

빈생성시 사용하는 <beans:beans> 와 <bean> 태그는 같은 태그이며 스케마를 생략할때 주로 <bean> 태그를 사용한다.

2. 시작페이지 만들기

학습 목표 : 매핑 이후 컨트롤러에서 뷰리졸브를 통하여 뷰에 출력할 수 있다.



스텝 1:

컨트롤러 작성하기

컨트롤러 생성

- 패키지 : com.springmvc.controller
- 파일명 : WelcomeController

BookMarket/src/main/java/com/springmvc/controller/WelcomeController.java

@Controller 클래스 위에 이거 있으면 자동으로 생성

```
public class WelcomeController {
```

@RequestMapping(value = "/home", method = RequestMethod.GET) 함수, 클래스 매핑 가능

```
public String welcome(Model model) {
```

model.addAttribute("greeting", "Welcome to BookMarket");

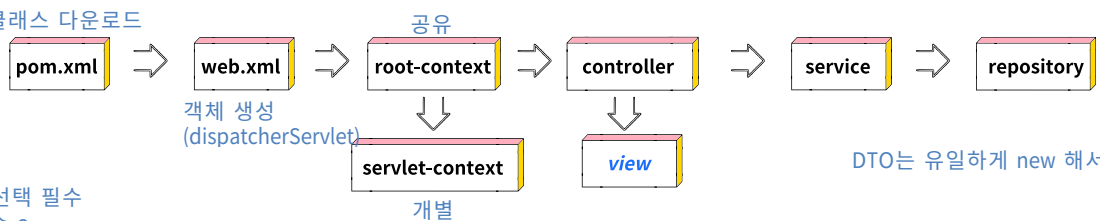
model.addAttribute("strapline", "Welcome to Web Shopping Mall!");

return "welcome";

```
}
```

```
}
```

클래스 다운로드



자바, 아파치톰캣, 스프링 버전 선택 필수
-> version마다 view가 달라질 수 o

apachTomcat 9.0 - spring 5
apachTomcat 10.0 - spring 6
(호환 안 됨)



스텝 2:

뷰 작성하기

뷰생성 및 CSS적용

- 뷰경로 : /WEB-INF/views/welcome.jsp
- CSS경로 : /resources/css/bootstrap.min.css

BookMarket/src/main/webapp/WEB-INF/views/welcome.jsp

```
<%@ page contentType="text/html; charset=utf-8"%>
```

```
<html>
```

```
<head>
```

```
<title>Welcome</title>
```

```
<link href="/resources/css/bootstrap.min.css" rel="stylesheet">
```

```
</head>
```

```
<body>
```

html: chrome 해석 가능
xml: chrome 해석 불가능

```
<nav class="navbar navbar-expand navbar-dark bg-dark">
  <div class="container">
    <div class="navbar-header">
      <a class="navbar-brand" href="/home">Home</a>
    </div>
  </div>
</nav>
```

1

```
<div class="jumbotron">
  <div class="container">
    <h1 class="display-3">${greeting}</h1>
  </div>
</div>
```

2

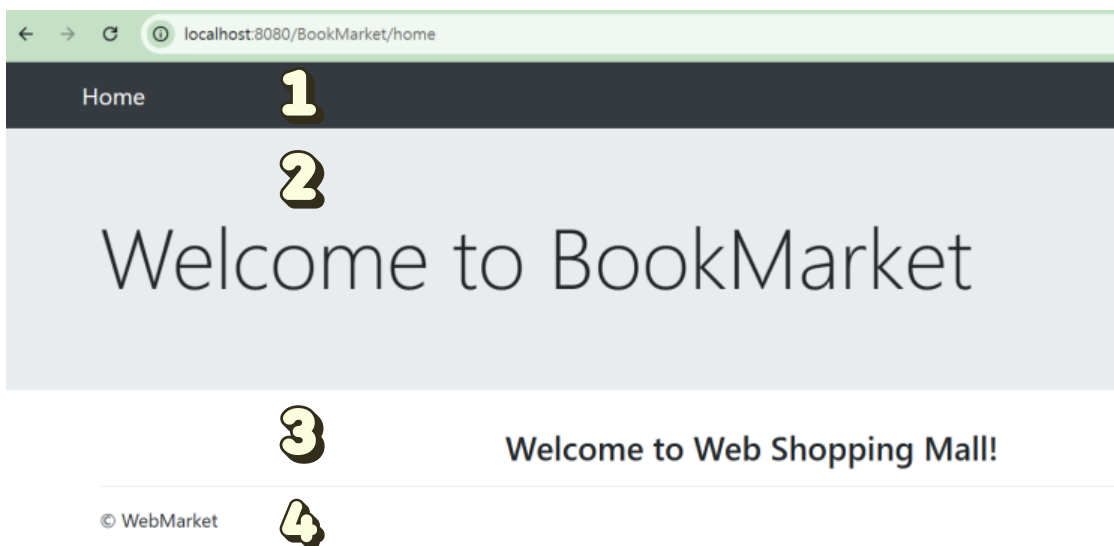
```
<div class="container">
  <div class="text-center">
    <h3>${strapline}</h3>
  </div>
</div>
```

3

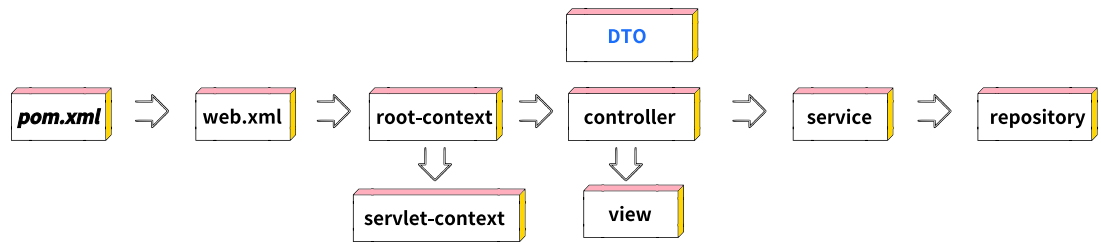
```
<footer class="container">
  <hr>
  <p>&copy; WebMarket</p>
</footer>
```

4

```
</body>
</html>
```



3. 전체 도서 목록 보기



스텝 1:

DTO 클래스 생성 및 코드 작성

DTO 클래스 생성

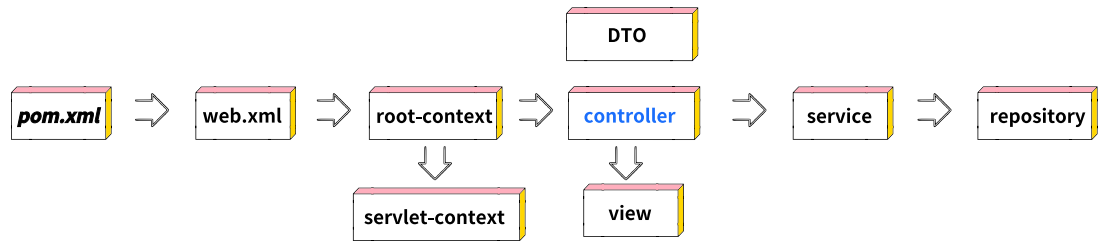
- 패키지 : com.springmvc.domain
- 파일이름 : Book

BookMarket/src/main/java/com/springmvc/domain/Book.java

```
public class Book {  
    private String bookId;  
    private String name;  
    private int unitPrice;  
    private String author;  
    private String description;  
    private String publisher;  
    private String category;  
    private long unitsInStock;  
    private String releaseDate;  
    private String condition;  
  
    // 기본생성자  
    // 일반생성자  
  
    // Getter , Setter  
}
```

위의 생성자와 Getter, Setter를 만들기 위해서는 소스코드에서 마우스오른쪽 선택 후

- 생성자 : Source > Generate Constructor using Fields를 선택
- Getter,Setter : Source > Generate Getters and Setters를 선택



📌 스텝 2: 컨트롤러 작성하기

컨트롤러 클래스 생성

- 패키지 : com.springmvc.controller
- 파일이름 : BookController

BookMarket/src/main/java/com/springmvc/controller/BookController.java

```

@Controller
public class BookController {
    @Autowired
    private BookService bookService;
    @RequestMapping(value = "/books", method = RequestMethod.GET)
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }
}

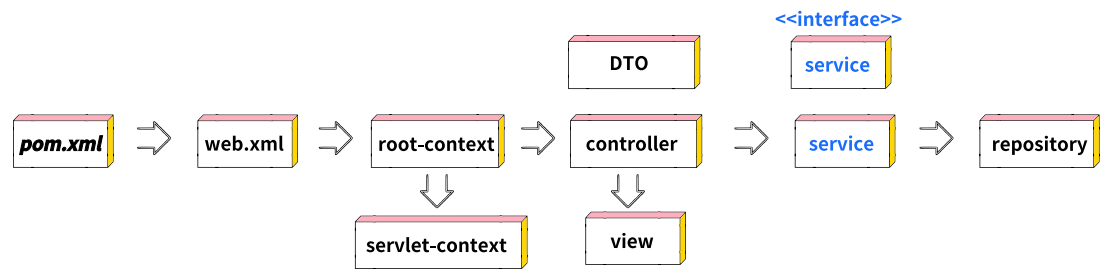
```

@Controller, @Autowired, @RequestMapping 애노테이션

- @Controller : Servlet-context에서 자동으로 객체가 생성된다.
- @Autowired : Servlet-context가 생성한 객체를 자동으로 주입(대입)한다.
- @RequestMapping : JSP의 @WebServlet("/book")와 같은 기능으로 매핑하여 실행한다.

주의사항

- Servlet-context.xml의 <context:component-scan>에서 @Controller와 같은 애노테이션이 객체가 생성되도록 패키지 지정이 =되었는지 반드시 확인해야한다.
- 에러가 발생하지 않도록 다음과 같이 코드를 수정한다.
 <context:component-scan base-package="com.springmvc.*" />



스텝 3: 서비스 클래스 작성하기

서비스 인터페이스와 클래스 작성하기

- 패키지 : com.springmvc.service
- 인터페이스 이름 : BookService.java
- 클래스 이름 : BookServiceImpl.java

BookMarket/src/main/java/com/springmvc/service/BookService.java

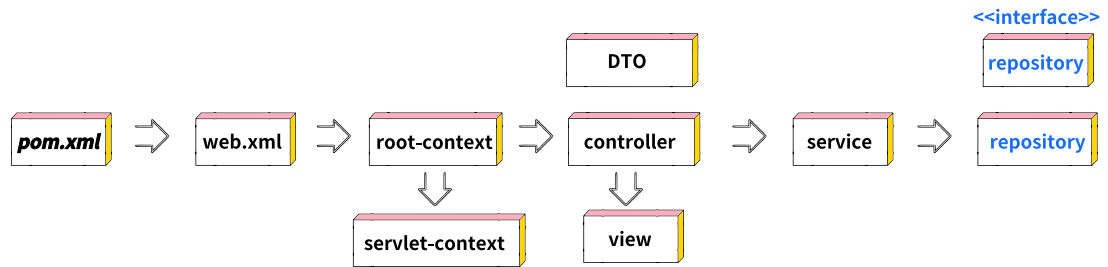
```
public interface BookService {
    List<Book> getAllBookList();
}
```

BookMarket/src/main/java/com/springmvc/service/BookServiceImpl.java

```
@Service
public class BookServiceImpl implements BookService {
    @Autowired
    private BookRepository bookRepository;
    public List<Book> getAllBookList() {
        return bookRepository.getAllBookList();
    }
}
```

interface를 구현(상속)해서 사용하는 이유는?

- 코드의 구조와 유지 관리성을 향상시키기 위해서 사용
- 다형성 : 인터페이스를 사용하면 서로 다른 클래스가 동일한 메서드를 정의하고 내용은 다르게 작성하여 객체를 교환 가능하게 만들 수 있다



🗨️ 스텝 4: 리파지토리 클래스 작성하기

리파지토리 인터페이스와 클래스 작성하기

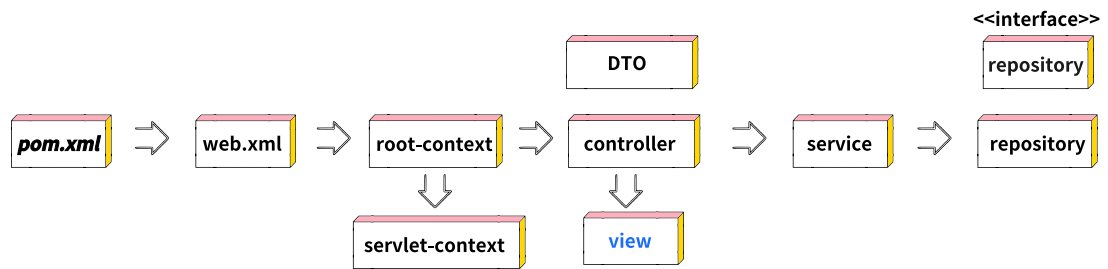
- 패키지 : com.springmvc.repository
- 인터페이스 이름 : BookRepository.java
- 클래스 이름 : BookRepositoryimpl.java

BookMarket/src/main/java/com/springmvc/repository/BookRepository.java

```
public interface BookRepository {
    List<Book> getAllBookList();
}
```

BookMarket/src/main/java/com/springmvc/repository/BookRepositoryimpl.java

```
@Repository
public class BookRepositoryImpl implements BookRepository{
    private List<Book> listOfBooks = new ArrayList<Book>();
    public BookRepositoryImpl() {
        Book book1 = new Book("ISBN1234", "C# 교과서", 30000);
        book1.setAuthor("박용준");
        book1.setDescription("C# 교과서...중략");
        book1.setPublisher("길벗");
        book1.setCategory("IT전문서");
        book1.setUnitsInStock(1000);
        book1.setReleaseDate("2020/05/29");
        Book book2 = new Book("ISBN1235", "Node.js 교과서", 36000);
        book2.setAuthor("조현영");
        book2.setDescription("이 책은 프론트부터 ...중략");
        book2.setPublisher("길벗");
        book2.setCategory("IT전문서");
        book2.setUnitsInStock(1000);
        book2.setReleaseDate("2020/07/25");
        //book3 생략
        listOfBooks.add(book1);
        listOfBooks.add(book2);
        listOfBooks.add(book3);
    }
    @Override
    public List<Book> getAllBookList() {
        return listOfBooks;
    }
}
```



스프링 MVC: 뷰 작성하기 : taglib 이용하기

뷰 작성하기

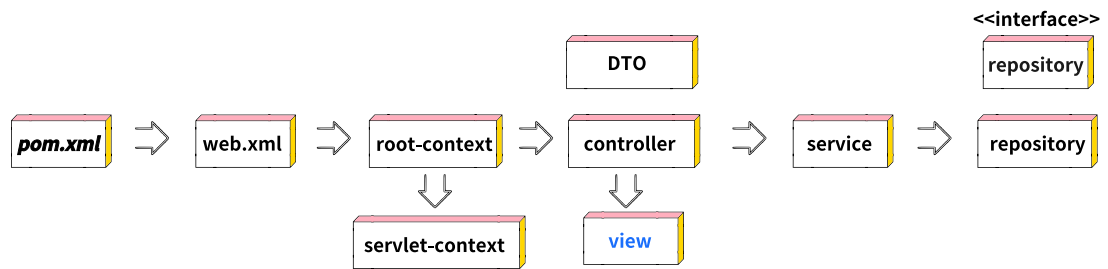
- 경로 : /WEB-INF/views/books.jsp

BookMarket/src/main/java/com/springmvc/repository/BookRepository.java

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions"%>
<html>
<head>
<link href="./resources/css/bootstrap.min.css" rel="stylesheet">
<title>도서 목록</title>
</head>
<body>
<nav class="navbar navbar-expand navbar-dark bg-dark">
<div class="container">
<div class="navbar-header">
<a class="navbar-brand" href="/home">Home</a>
</div>
</div>
</nav>
<div class="jumbotron">
<div class="container">
<h1 class="display-3">도서 목록</h1>
</div>
</div>
<div class="container">
<div class="row" align="center">
<c:forEach items="${bookList}" var="book">
<div class="col-md-4">
<h3>${book.name}</h3>
<p>${book.author}
<br> ${book.publisher} | ${book.releaseDate}
<p align="left">${fn:substring(book.description, 0, 100)}...
<p>${book.unitPrice}원
</div>
</c:forEach>
</div>
<hr>
<footer>
<p>&copy; BookMarket</p>
</footer>
</div>
</body>
</html>

```



스텝 5: 뷰 작성하기 : 일반태그 이용하기

뷰 작성하기

- 경로 : /WEB-INF/views/books.jsp

BookMarket/src/main/java/com/springmvc/repository/BookRepository.java

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ page import="com.springmvc.domain.Book" %>
<%@ page import="java.util.*" %>
<% ArrayList<Book> bookList = (ArrayList<Book>) request.getAttribute("bookList");%>
<html>
<head>
<link href="./resources/css/bootstrap.min.css" rel="stylesheet">
<title>도서 목록</title>
</head>
<body>
    <nav class="navbar navbar-expand navbar-dark bg-dark">
        <div class="container">
            <div class="navbar-header">
                <a class="navbar-brand" href="/home">Home</a>
            </div>
        </div>
    </nav>
    <div class="jumbotron">
        <div class="container">
            <h1 class="display-3">도서 목록</h1>
        </div>
    </div>
    <div class="container">
        <div class="row" align="center">
            <% for(Book book : bookList) { %>
                <div class="col-md-4">
                    <h3><%=book.getName()%></h3>
                    <p><%=book.getAuthor() %>
                        <br> <%=book.getPublisher() %> | <%=book.getReleaseDate() %>
                    <p align="left"><%=book.getDescription().substring(0,20) %>...
                    <p><%=book.getUnitPrice() %>원
                </div><%> %>
            </div><hr>
        <footer>
            <p>&copy; BookMarket</p>
        </footer>
    </div>
</body>
</html>

```


← → ↺ localhost:8080/BookMarket/books

🔍 ☆ 🌐 🖨️ 🚦

Home

도서 목록

C# 교과서

박용준
길벗 | 2020/05/29

C# 교과서』는 생애 첫 프로그래밍 언어로 C#을 시작하는 독자를 대상으로 한다. 특히 응용 프로그래머를 위한 C# 입문서로, C#을 사용하여 게임(유니티), 웹, 모바일, IoT...

30000원

Node.js 교과서

조현영
길벗 | 2020/07/25

이 책은 프런트부터 서버, 데이터베이스, 배포까지 아우르는 광범위한 내용을 다룬다. 군더더기 없는 직관적인 설명으로 기본 개념을 확실히 이해하고, 노드의 기능과 생태계를 사용해보면...

36000원

어도비 XD CC 2020

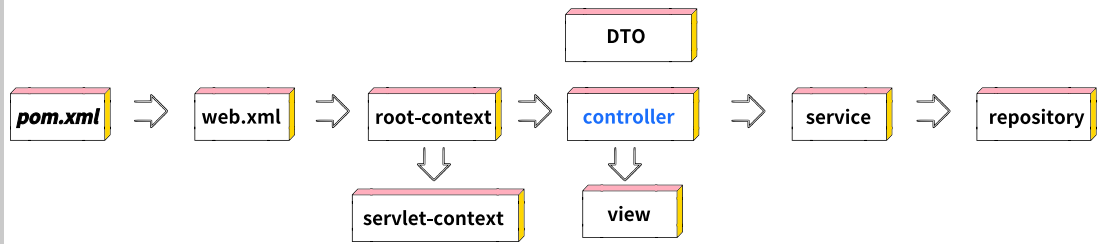
김두한
길벗 | 2019/05/29

어도비 XD 프로그램을 통해 UI/UX 디자인을 배우고자 하는 예비 디자이너의 눈높이에 맞게 기본적인 도구를 활용한 아이콘 디자인과 웹&앱 페이지 디자인, UI 디자인, 앱 디자인...

25000원

© BookMarket

4. 컨트롤러 구성하기



스텝 1:

컨트롤러 매핑 수정

컨트롤러 클래스 수정

- 패키지 : com.springmvc.controller
- 파일이름 : BookController

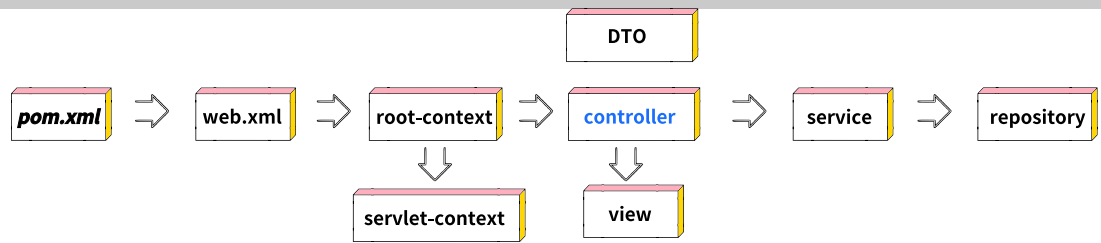
BookMarket/src/main/java/com/springmvc/controller/BookController.java

```
@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @RequestMapping
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }
}
```

유저가 `http://localhost:8080/BookMarket/books` 를 요청했을 경우 아래의 애노테이션에 매핑되며 해당 매핑은 다음과 같습니다.

- `@RequestMapping("/books")` : 클래스위에 선언되어 클래스 단위 매핑이라고한다.
- `@RequestMapping` : 클래스 매핑 이후에 매핑되는 메서드가 없으며 기본적으로 매핑된다.



스텝 2: 컨트롤러에서 RequestMapping 수정하기

컨트롤러 클래스 수정

- 패키지 : com.springmvc.controller
- 파일이름 : BookController

BookMarket/src/main/java/com/springmvc/controller/BookController.java

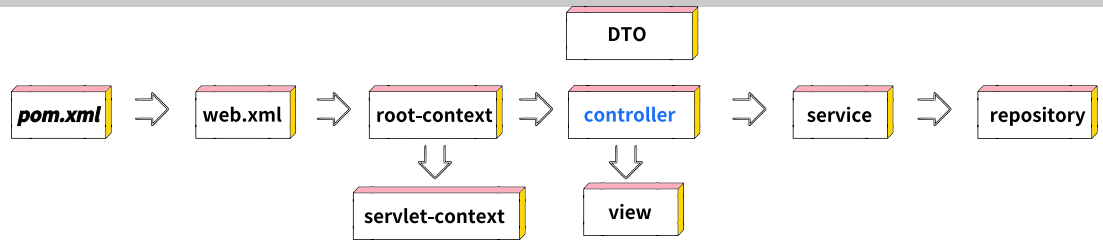
```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @RequestMapping("/all")
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }
}
  
```

유저가 `http://localhost:8080/BookMarket/books/all` 를 요청했을 경우 아래의 애노테이션에 매핑되며 해당 매핑은 다음과 같습니다.

- `@RequestMapping("/books")` : 클래스위에 선언되어 클래스 단위 매핑이라고한다.
- `@RequestMapping("/all")` : 클래스 매핑 이후에 매핑되는 메서드이다.
- 만약 `http://localhost:8080/BookMarket/books` 로 요청이 들어오면 기본 매핑이 없으므로 아무것도 매핑되지 않고 404 에러를 전송한다.



스텝 5 : 컨트롤러에서 RequestMapping 단순화하기

컨트롤러 클래스 수정

- 패키지 : com.springmvc.controller
- 파일이름 : BookController

BookMarket/src/main/java/com/springmvc/controller/BookController.java

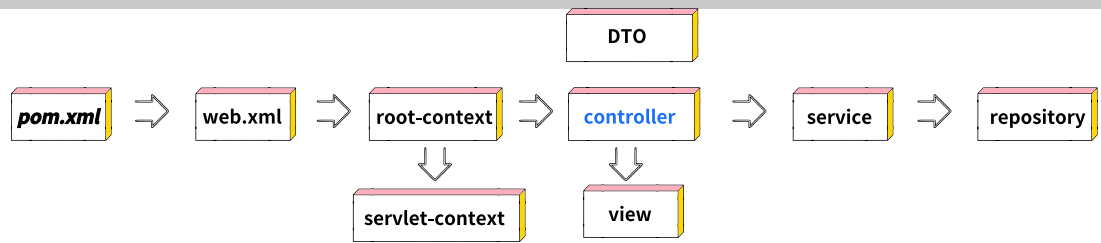
```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @GetMapping
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }
    /*
    @RequestMapping("/all")
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }
    */
}
  
```

유저가 http://localhost:8080/BookMarket/books 를 요청했을경우 아래의 애노테이션에 매핑되며 해당 매핑은 다음과 같습니다.

- 요청이 들어오면 기본 @GetMapping에 매치되어 동작합니다.



스텝 4: 컨트롤러에서 ModelAndView 사용하여 결과 반환하기

컨트롤러 클래스 수정

- 패키지 : com.springmvc.controller
- 파일이름 : BookController

BookMarket/src/main/java/com/springmvc/controller/BookController.java

```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @GetMapping
    public String requestBookList(Model model) {
        List<Book> list = bookService.getAllBookList();
        model.addAttribute("bookList", list);
        return "books";
    }

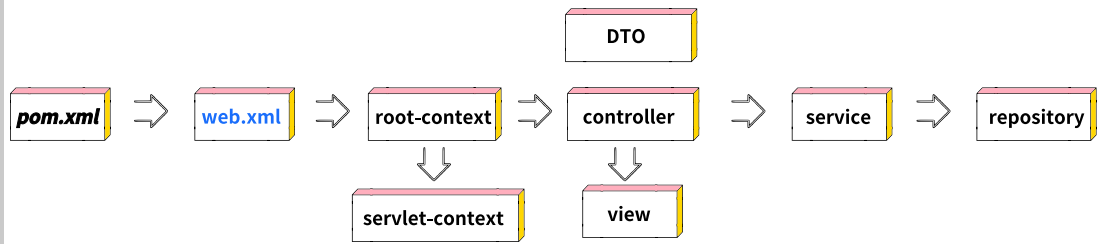
    @GetMapping("/all")
    public ModelAndView requestAllBooks() {
        ModelAndView modelAndView = new ModelAndView();
        List<Book> list = bookService.getAllBookList();
        modelAndView.addObject("bookList", list);
        modelAndView.setViewName("books");
        return modelAndView;
    }
}

```

유저가 http://localhost:8080/BookMarket/books/all 를 요청했을 경우 requestAllBooks() 메서드에 매핑됩니다.

- ModelAndView 객체는 전달할 Model 객체와 View정보를 같이 담는 개체입니다.
- 반드시 객체를 수동으로 생성이후 사용가능합니다.
- DTO를 addObject()메서드를 사용하여 key,value형태로 저장합니다.
- View의 이름을 setViewName()메서드를 사용하여 문자열 형태로 저장합니다.

5. 컨트롤러 전처리: URL 파라미터 수신하기



스텝 1:

한글 깨지지 않게 필터 인코딩 처리하기

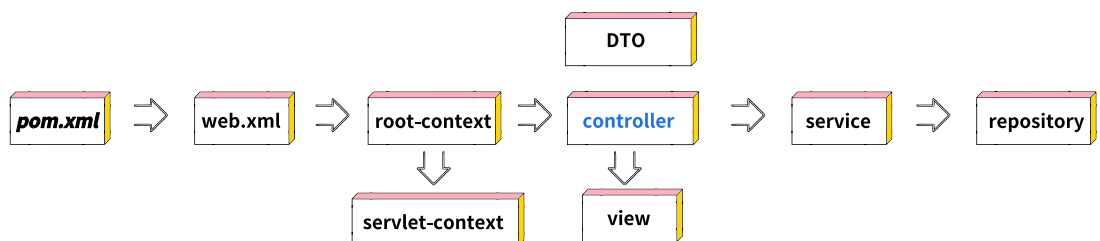
web.xml을 통하여 서블릿 실행이전에 인코딩 객체가 동작하여 한글 인코딩을 실시하도록한다.

- 파일위치 : /WEB-INF/web.xml

BookMarket/src/main/webapp/WEB-INF/web.xml

```

<filter>
  <filter-name>encodingFilter</filter-name>
  <filter-class>org.springframework.web.filter.CharacterEncodingFilter
</filter-class>
  <init-param>
    <param-name>encoding</param-name>
    <param-value>UTF-8</param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>encodingFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
  
```



스텝 2:

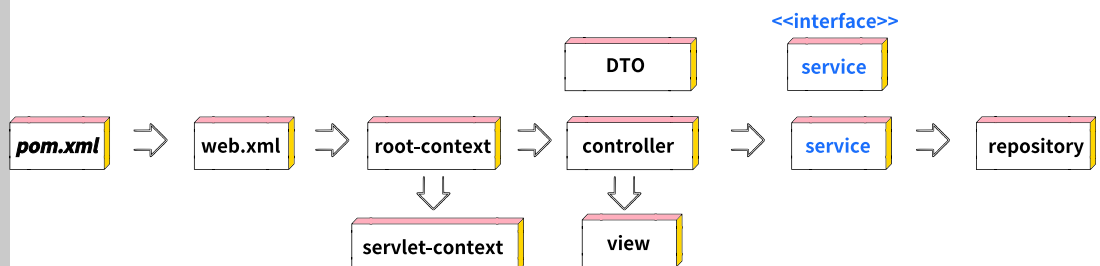
컨트롤로에서 파라미터 처리 및 메서드 구현

클라이언트에서 "http://localhost:8080/BookMarket/books/IT전문서" 를 요청하면 다음과 같이 처리된다.

- 클래스 매핑 : /books
- 파라미터 매핑 : String bookCategory = "IT전문서"

- `@GetMapping("/{category}") : @PathVariable("category")`와 매핑되며 "IT전문서" URI는 `String bookCategory` 변수에 대입됩니다.
- URL의 일부를 데이터 처럼 활용할수 있는 방법입니다.

- `<a href="books?category=IT활용서">` 태그를 활용하여 파라미터를 전달하는 방법
- `<form><input type="text" value="IT활용서" name="category">` 폼을 이용하여 파라미터를 전달하는 방법
- `"http://localhost:8080/BookMarket/books/IT전문서"` URL 일부를 이용하여 파라미터를 전달하는 방법



서비스 클래스 수정하기

- 패키지 : com.springmvc.service
- 파일이름 : BookService.java

BookMarket/src/main/java/com/springmvc/service/BookService.java

```
public interface BookService {  
    List<Book> getAllBookList();  
    List<Book> getBookListByCategory(String category);  
}
```

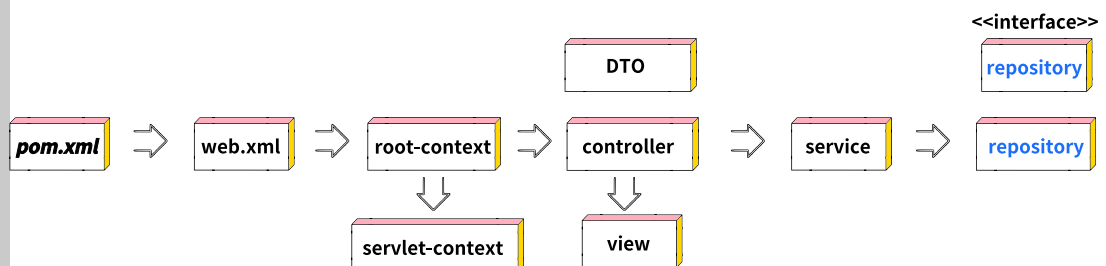
List<Book> getBookListByCategory(String category); 함수를 추가합니다.

BookMarket/src/main/java/com/springmvc/service/BookServiceImpl.java

```
@Service  
public class BookServiceImpl implements BookService{  
    @Autowired  
    private BookRepository bookRepository;  
  
    public List<Book> getBookListByCategory(String category) {  
        List<Book> booksByCategory = bookRepository.getBookListByCategory(category);  
        return booksByCategory;  
    }  
}
```

public List<Book> getBookListByCategory(String category){} 메서드를 추가합니다.

현재 서비스 클래스는 중계역할만 할 뿐입니다.



스텝 4:

리포지토리 클래스 수정하기

리포지토리 클래스 수정

- 패키지 : com.springmvc.service
- 파일이름 : BookRepository.java

BookMarket/src/main/java/com/springmvc/repository/BookRepository.java

```
public interface BookRepository {  
    List<Book> getAllBookList();  
    List<Book> getBookListByCategory(String category);  
}
```

List<Book> getBookListByCategory(String category){} 메서드를 추가합니다.

BookMarket/src/main/java/com/springmvc/repository/BookRepositoryImpl.java

```
@Repository
public class BookRepositoryImpl implements BookRepository{
    private List<Book> listOfBooks = new ArrayList<Book>();

    public List<Book> getBookListByCategory(String category) {
        List<Book> booksByCategory = new ArrayList<Book>();
        for(int i=0 ; i<listOfBooks.size() ; i++) {
            Book book = listOfBooks.get(i);
            if(category.equalsIgnoreCase(book.getCategory()))
                booksByCategory.add(book);
        }
        return booksByCategory;
    }
}
```

Repository는 DTO단위로 데이터를 처리하는 경향이 강한 클래스입니다.

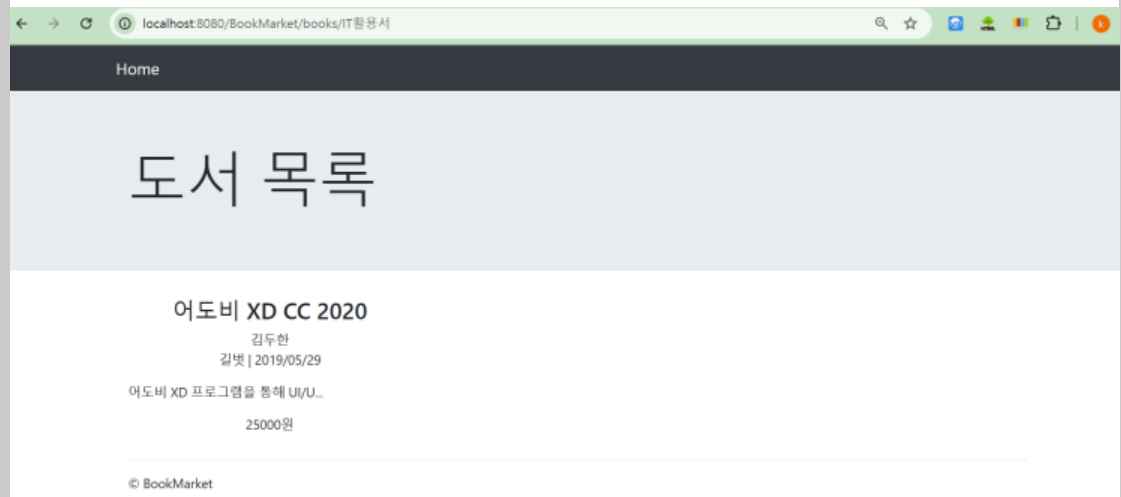
- getBookListByCategory() 메서드는 현재 데이터베이스 역할을 하는 listOfBooks에서 DTO를 하나씩 꺼내와서 파라미터 category와 같은 카테고리인지 검사하며 같은 카테고리는 booksByCategory 변수에 담아서 최종적으로 리턴합니다.



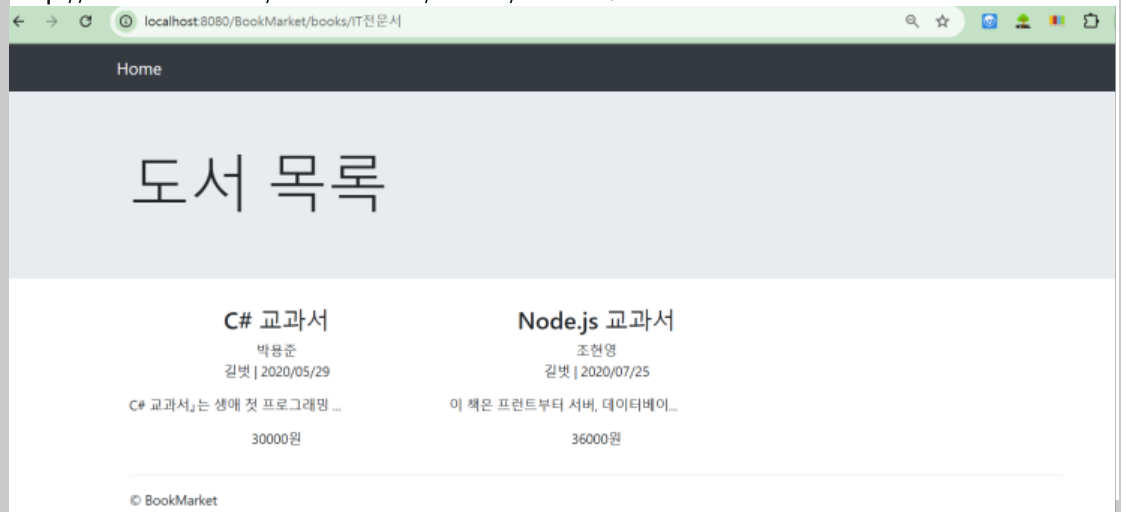
스텝 5:

결과 확인하기

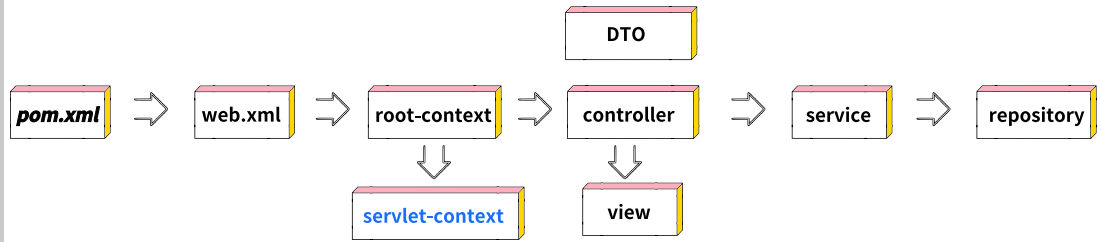
<http://localhost:8080/BookMarket/books/IT활용서>



<http://localhost:8080/BookMarket/books/IT전문서>



6. 컨트롤러 전처리: 매트릭스 파라미터



스텝 1:

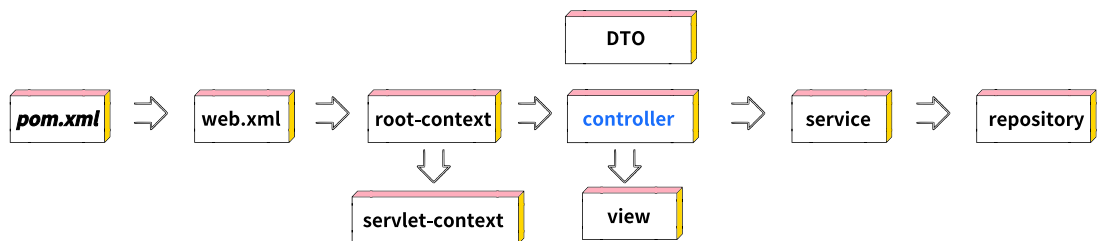
매트릭스 변수를 사용하도록 객체를 수정

servlet-context.xml을 통하여 생성된 핸들러 매핑과 핸들러 아답터의 객체에서 매트릭스 기호를 인식하도록 코드를 수정.

- 파일위치 : /WEB-INF/spring/appServlet/servlet-context.xml

BookMarket/src/main/webapp/WEB-INF/spring/appServlet/servlet-context.xml

```
<annotation-driven enable-matrix-variables="true" />
```



스텝 2:

컨트롤로에서 파라미터 처리 및 메서드 구현

클라이언트에서 "http://localhost:8080/BookMarket/books/fliter/bookFilter;publisher=길벗;category=IT전문서" 를 요청하면 다음과 같이 처리된다.

- 클래스 매핑 : /books
- 메서드 매핑 : /filter/{bookFilter}
- 파라미터 : ;publisher=길벗;category=IT전문서

BookMarket/src/main/java/com/springmvc/controller/BookController.java

```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @GetMapping("/filter/{bookFilter}")
    public String requestBooksByFilter(
        @MatrixVariable(pathVar="bookFilter") Map<String,List<String>> bookFilter,
        Model model) {
        Set<Book> booksByFilter = bookService.getBookListByFilter(bookFilter);
        model.addAttribute("bookList", booksByFilter);
        return "books";
    }
}

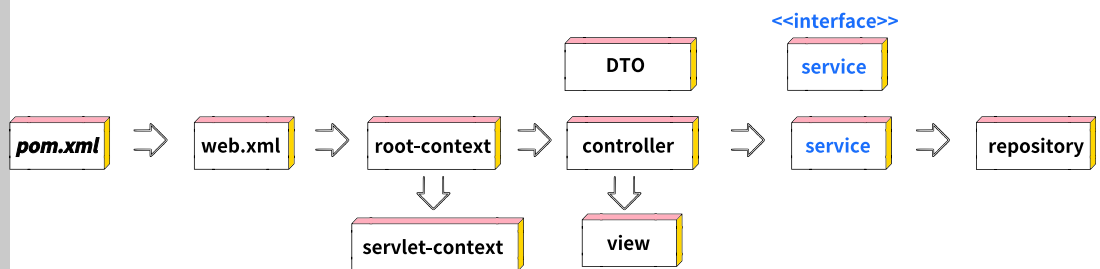
```

bookFilter 뒤의 세미콜론(;) 부터 파라미터로 인식하며 Map<key,value> 형태로 저장된다.
즉, 다음 URL의 형식으로 저장되는 키와 value는 다음과 같다.

"http://localhost:8080/BookMarket/books/fliter/bookFilter
;publisher=길벗;category=IT전문서"

- key : publisher , value : 길벗
- key : category , value : IT전문서

만약 IT전문서,IT학습서 2개의 value가 전달될 수 있기때문에 value는 List<String>형태로 저장한다



스텝 5 :

서비스 클래스 수정하기

서비스 클래스 수정

- 패키지 : com.springmvc.service
- 파일이름 : BookService.java

BookMarket/src/main/java/com/springmvc/service/BookService.java

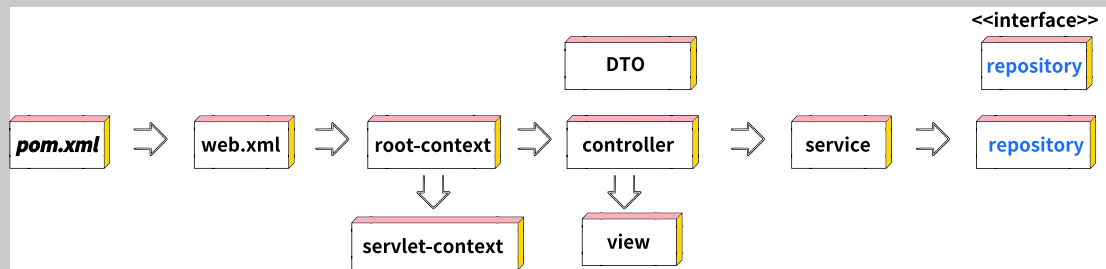
```
public interface BookService {
    Set<Book> getBookListByFilter(Map<String, List<String>> filter);
}
```

List<Book> getBookListByFilter(); 함수를 추가합니다.

BookMarket/src/main/java/com/springmvc/service/BookServiceImpl.java

```
@Service
public class BookServiceImpl implements BookService{
    @Autowired
    private BookRepository bookRepository;

    public Set<Book> getBookListByFilter(Map<String, List<String>> filter) {
        Set<Book> booksByFilter = bookRepository.getBookListByFilter(filter);
        return booksByFilter;
    }
}
```



스텝 4:

리포지토리 클래스 수정하기

리포지토리 클래스 수정

- 패키지 : com.springmvc.service
- 파일이름 : BookRepository.java

BookMarket/src/main/java/com/springmvc/repository/BookRepository.java

```
public interface BookRepository {
    Set<Book> getBookListByFilter(Map<String, List<String>> filter);
}
```

List<Book> getBookListByFilter(){} 메서드를 추가합니다.

BookMarket/src/main/java/com/springmvc/repository/BookRepositoryimpl.java

```
@Repository
public class BookRepositoryImpl implements BookRepository{
    private List<Book> listOfBooks = new ArrayList<Book>();

    public Set<Book> getBookListByFilter(Map<String, List<String>> filter) {
        Set<Book> booksByPublisher = new HashSet<Book>();
        Set<Book> booksByCategory = new HashSet<Book>();
        Set<String> booksByFilter = filter.keySet();
        if (booksByFilter.contains("publisher")) {
            for (int j = 0; j < filter.get("publisher").size(); j++) {
                String publisherName = filter.get("publisher").get(j);
                for (int i = 0; i < listOfBooks.size(); i++) {
                    Book book = listOfBooks.get(i);
                    if (publisherName.equalsIgnoreCase(book.getPublisher()))
                        booksByPublisher.add(book);
                }
            }
        }
        if (booksByFilter.contains("category")) {
            for (int i = 0; i < filter.get("category").size(); i++) {
                String category = filter.get("category").get(i);
                List<Book> list = getBookListByCategory(category);
                booksByCategory.addAll(list);
            }
        }
        booksByCategory.retainAll(booksByPublisher);
        return booksByCategory;
    }
}
```

http://localhost:8080/BookMarket/books/filter
/bookFilter;publisher=길벗;category=IT전문서

위와 같이 요청이 발생하면 파라미터를 수신하는 부에서 Map은 아래와 같이 생성된다.

Key : String	Value :List<String>
publisher	길벗
category	IT전문서

```
public Set<Book> getBookListByFilter(Map<String, List<String>> filter) {  
    Set<Book> booksByPublisher = new HashSet<Book>();  
    Set<Book> booksByCategory = new HashSet<Book>();  
    Set<String> booksByFilter = filter.keySet(); //keySet()을 통해 전체 키를 꺼내서 담음  
    if (booksByFilter.contains("publisher")) { //booksByFilter에 "publisher"가 있다면  
        for (int j = 0; j < filter.get("publisher").size(); j++) { //publisher 키 한개 1번 동작  
            String publisherName = filter.get("publisher").get(j); // 길벗이 저장됨  
            for (int i = 0; i < listOfBooks.size(); i++) { // DB역할을 하는 listOfBooks 3번동작  
                Book book = listOfBooks.get(i); // 첫번째 DTO 꺼냄  
                if (publisherName.equalsIgnoreCase(book.getPublisher())) // 길벗과 같으면  
                    booksByPublisher.add(book); //booksByPublisher에 담음 3개다 길벗임  
            }  
        }  
    }  
    if (booksByFilter.contains("category")) {  
        for (int i = 0; i < filter.get("category").size(); i++) {  
            String category = filter.get("category").get(i); //IT전문서  
            List<Book> list = getBookListByCategory(category);  
            booksByCategory.addAll(list); // 2개만 IT전문서  
        }  
    }  
    booksByCategory.retainAll(booksByPublisher);  
    return booksByCategory;  
}
```

retainAll함수를 통하여 교집합을 실시하면 C#, Node.js 두과목 외 다른 DTO는 삭제된다.

다만 여기서는 삭제되는 항목은 없다. 즉 둘다 가지고 있어야지만 전달할 수 있다. (AND 조건)

booksByPublisher

C# 교과서	어드비 XD CC 2020	Node.js 교과서
이종훈 출판 2020/05/29	김두현 출판 2019/05/29	조현영 출판 2020/07/25
C# 교과서는 어떻게 프로그래밍 언어로 C#을 시작하는 독자를 대상으로 한다. 특히 공통 프로그래밍 언어를 위한 C# 입문서로, C#을 사용하여 개발한다. 이는, 웹, 모바일, IoT...	어드비 XD 프로그래밍 언어로 XD를 시작하는 독자를 대상으로 한다. 특히 공통 프로그래밍 언어를 위한 XD 입문서로, XD를 사용하여 개발한다. 이는, 웹, 모바일, IoT...	이 책은 프론트엔드 서버, 데이터베이스, 백엔드까지 아우르는 종합적인 내용을 다룬다. 군더더기 없는 직관적인 설명으로 기본 개념을 확실히 이해하고, Node.js 기능과 생태계를 이해해보면...
30000원	25000원	36000원
상세정보 >	상세정보 >	상세정보 >

booksByCategory

C# 교과서	Node.js 교과서
이종훈 출판 2020/05/29	조현영 출판 2020/07/25
C# 교과서는 어떻게 프로그래밍 언어로 C#을 시작하는 독자를 대상으로 한다. 특히 공통 프로그래밍 언어를 위한 C# 입문서로, C#을 사용하여 개발한다. 이는, 웹, 모바일, IoT...	이 책은 프론트엔드 서버, 데이터베이스, 백엔드까지 아우르는 종합적인 내용을 다룬다. 군더더기 없는 직관적인 설명으로 기본 개념을 확실히 이해하고, Node.js 기능과 생태계를 이해해보면...
30000원	36000원
상세정보 >	상세정보 >

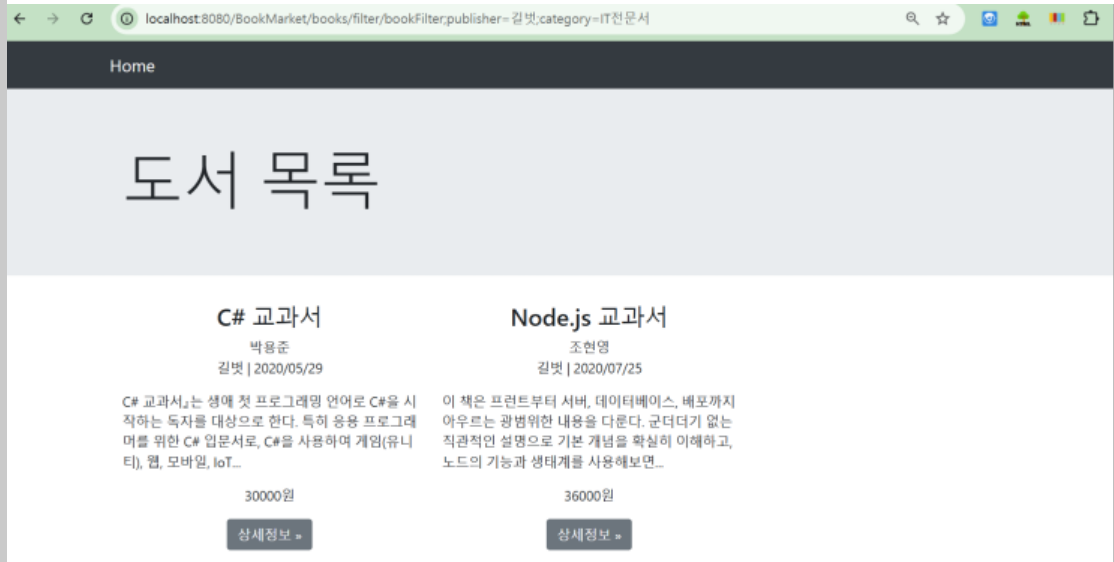


스텝 5:

실행결과

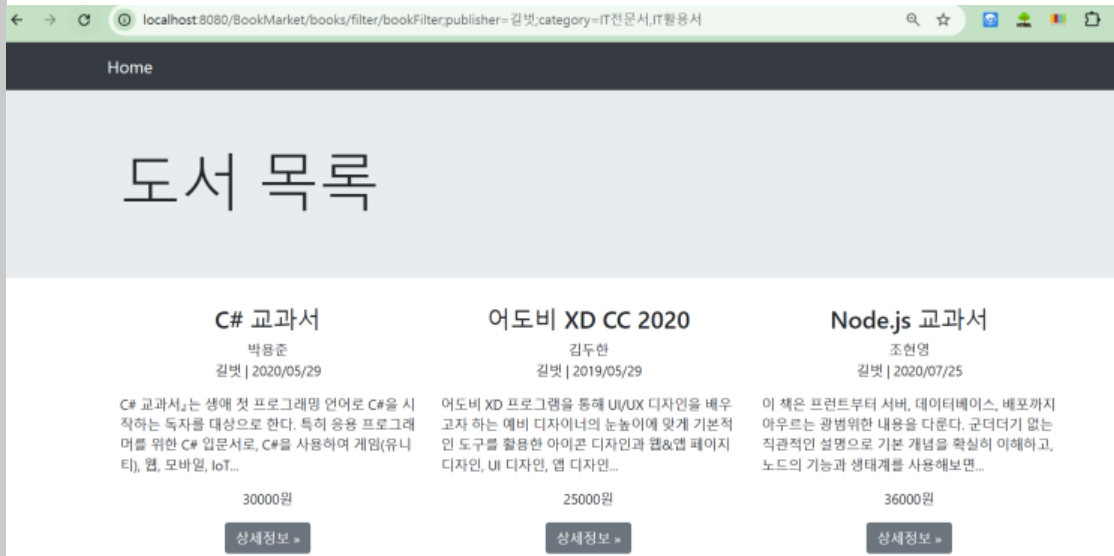
요청 URL

- `http://localhost:8080/BookMarket/books/filter/bookFilter; publisher=길벗;category=IT전문서`

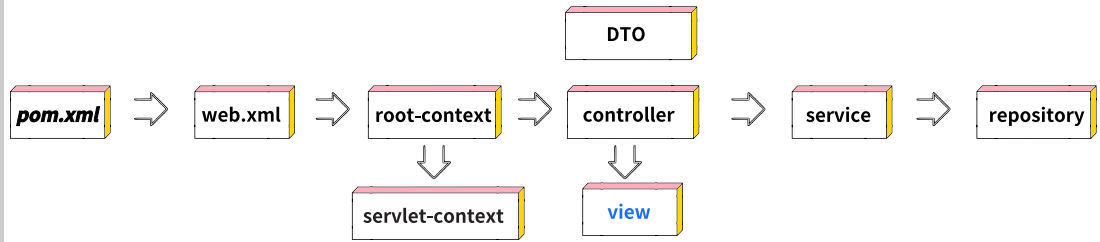


요청 URL

- `http://localhost:8080/BookMarket/books/filter/bookFilter; publisher=길벗;category=IT전문서,IT활용서`



7. 컨트롤러 전처리: 리퀘스트 파라미터



스텝 1:

요청 생성하기

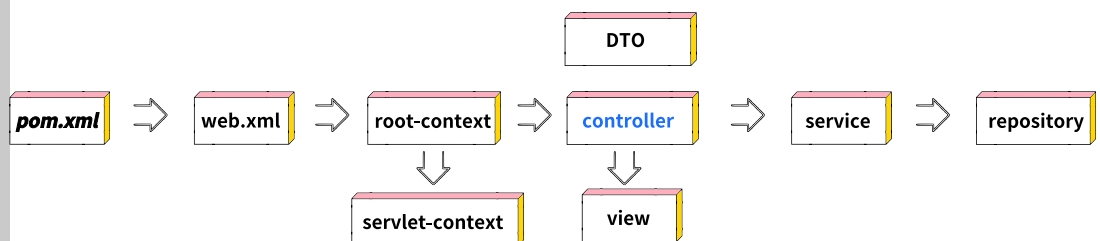
books.jsp에서 <a> 태그를 클릭하면 요청이 발생하도록 다음과 같이 코드를 추가 하도록하자.

- 파일위치 : /WEB-INF/views/books.jsp

BookMarket/src/main/webapp/WEB-INF/views/books.jsp

```

<c:forEach items="${bookList}" var="book">
  <div class="col-md-4">
    <h3>${book.name}</h3>
    <p>${book.author}
    <br> ${book.publisher} | ${book.releaseDate}
    <p align=left>${fn:substring(book.description, 0, 100)}...
    <p>${book.unitPrice}원
    <p>
    <a href="books/book?id=${book.bookId}" class="btn btn-Secondary" role="button">
      상세정보 &raquo;
    </a>
  </div>
</c:forEach>
  
```



스텝 2:

컨트롤로에서 파라미터 처리 및 메서드 구현

클라이언트에서 "http://localhost:8080/BookMarket/books/book?id=ISBN1234를 요청하면 다음과 같이 처리된다.

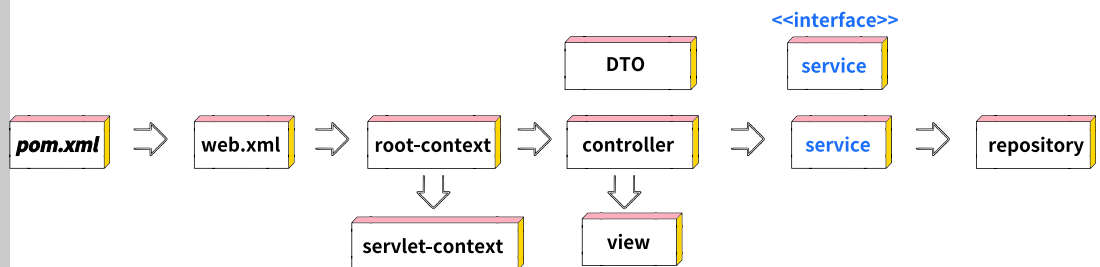
- 클래스 매핑 : /books
- 메서드 매핑 : /book
- 파라미터 : ?id=ISBN1234

BookMarket/src/com/springmvc/controller/BookController.java

```
@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @GetMapping("/book")
    public String requestBookById(@RequestParam("id") String bookId, Model model) {
        Book bookById = bookService.getBookById(bookId);
        model.addAttribute("book", bookById);
        return "book";
    }
}
```

클라이언트에서 "http://localhost:8080/BookMarket/books/book?id=ISBN1234"를 요청 String bookId 매개변수에 @RequestParam을 통하여 ISBN1234 파라미터가 대입된다.



스텝 5

서비스 클래스 수정하기

서비스 클래스 수정

- 패키지 : com.springmvc.service
- 파일이름 : BookService.java

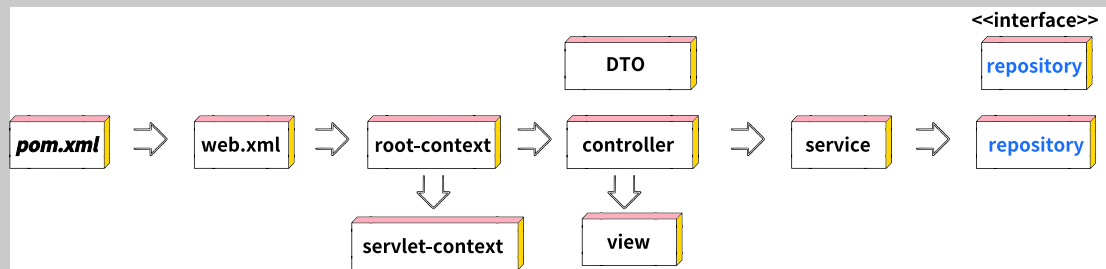
BookMarket/src/com/springmvc/service/BookService.java

```
public interface BookService {
    Book getBookById(String bookId);
}
```

BookMarket/src/com/springmvc/service/BookServiceImpl.java

```
@Service
public class BookServiceImpl implements BookService{
    @Autowired
    private BookRepository bookRepository;

    public Book getBookById(String bookId) {
        Book bookById = bookRepository.getBookById(bookId);
        return bookById;
    }
}
```

스텝 4:

리포지토리 클래스 수정하기

리포지토리 클래스 수정

- 패키지 : com.springmvc.repository
- 파일이름 : BookRepository.java

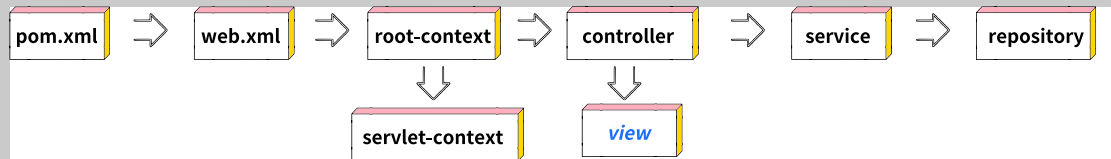
BookMarket/src/com/springmvc/repository/BookRepository.java

```
public interface BookRepository {
    Book getBookById(String bookId);
}
```

BookMarket/src/com/springmvc/repository/BookRepositoryimpl.java

```
@Repository
public class BookRepositoryImpl implements BookRepository{
    private List<Book> listOfBooks = new ArrayList<Book>();

    public Book getBookById(String bookId) {
        Book bookInfo = null;
        for(int i =0 ;i<listOfBooks.size(); i++) {
            Book book = listOfBooks.get(i);
            if (book!=null && book.getBookId()!=null && book.getBookId().equals(bookId)){
                bookInfo = book;
                break;
            }
        }
        if(bookInfo == null){
            throw new IllegalArgumentException("도서 ID가 "+bookId
                + "인 해당 도서를 찾을 수 없습니다.");
        }
        return bookInfo;
    }
}
```



스텝 5:

뷰 생성하기

뷰 생성하기

- 파일위치 : /WEB-INF/views/book.jsp

BookMarket/src/main/webapp/WEB-INF/views/book.jsp

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
<head>
<link href="<c:url value="/resources/css/bootstrap.min.css"/>"
      rel="stylesheet">
<title>도서 상세 정보</title>
</head>
<body>
  <nav class="navbar navbar-expand navbar-dark bg-dark">
    ...생략...
  </nav>
  <div class="jumbotron">
    <div class="container">
      <h1 class="display-3">도서 정보</h1>
    </div>
  </div>
  <div class="container">
    <div class="row">
      <div class="col-md-12">
        <h3>${book.name}</h3>
        <p>${book.description}</p>
        <br>
        <p><b>도서코드 : </b><span class="badge badge-info">${book.bookId}</span></p>
        <p><b>저자</b> : ${book.author}</p>
        <p><b>출판사</b> : ${book.publisher}</p>
        <p><b>출판일</b> : ${book.releaseDate}</p>
        <p><b>분류</b> : ${book.category}</p>
        <p><b>재고수</b> : ${book.unitsInStock}</p>
        <h4>${book.unitPrice}원</h4>
        <br>
        <p><a href="#" class="btn btn-primary">도서주문 &raquo;</a>
        <a href="books" class="btn btn-secondary">도서 목록 &raquo;</a>
      </div>
    </div>
  </div>
  <hr>
  <footer>
    <p>&copy; BookMarket </p>
  </footer>
</div>
</body>
</html>

```

localhost:8080/BookMarket/books



Home

도서 목록

C# 교과서

박용준
김벗 | 2020/05/29

C# 교과서」는 생애 첫 프로그래밍 언어로 C#을 시작하는 독자를 대상으로 한다. 특히 응용 프로그래머를 위한 C# 입문서로, C#을 사용하여 게임(유니티), 웹, 모바일, IoT...

30000원

상세정보 >

Node.js 교과서

조현영
김벗 | 2020/07/25

이 책은 프런트부터 서버, 데이터베이스, 배포까지 아우르는 광범위한 내용을 다룬다. 군더더기 없는 직관적인 설명으로 기본 개념을 확실히 이해하고, 노드의 기능과 생태계를 사용해보면...

36000원

상세정보 >

어도비 XD CC 2020

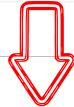
김두한
김벗 | 2019/05/29

어도비 XD 프로그램을 통해 UI/UX 디자인을 배우고자 하는 예비 디자이너의 눈높이에 맞게 기본적인 도구를 활용한 아이콘 디자인과 웹&앱 페이지 디자인, UI 디자인, 앱 디자인...

25000원

상세정보 >

© BookMarket



localhost:8080/BookMarket/books/book?id=ISBN1234

Home

도서 정보

C# 교과서

C# 교과서」는 생애 첫 프로그래밍 언어로 C#을 시작하는 독자를 대상으로 한다. 특히 응용 프로그래머를 위한 C# 입문서로, C#을 사용하여 게임(유니티), 웹, 모바일, IoT 등을 개발할 때 필요한 C# 기초 문법을 익히고 기본기를 탄탄하게 다지는 것이 목적이다.

도서코드 : ISBN1234

저자 : 박용준

출판사 : 김벗

출판일 : 2020/05/29

분류 : IT전문서

재고수 : 1000

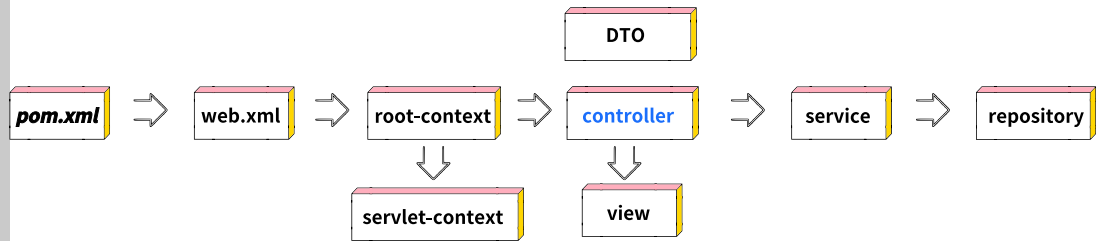
30000원

도서주문 >

도서 목록 >

© BookMarket

8. 도서등록 폼 페이지 제공하기



스텝 1:

컨트롤러에서 폼페이지 제공하기

클라이언트에서 "http://localhost:8080/BookMarket/books/add"를 요청하면 다음과 같이 폼페이지를 제공한다.

- 클래스 매핑 : /books
- 메서드 매핑 : /add
- 파라미터 : 없음

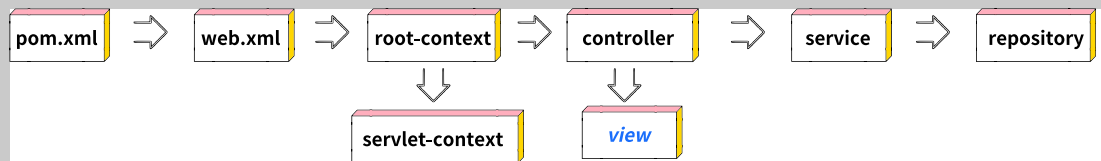
BookMarket/src/com/springmvc/controller/BookController.java

```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;
    //Case 1
    @GetMapping("/add")
    public String requestAddBookForm(
        @ModelAttribute("NewBook") Book book, Model model) {
        model.addAttribute("addTitle", "신규 도서 등록");
        return "addBook";
    }
    //Case 2
    @GetMapping("/add")
    public String requestAddBookForm(Model model) {
        model.addAttribute("book", new Book());
        model.addAttribute("addTitle", "신규 도서 등록");
        return "addBook";
    }
}
  
```

요청에 의해서 컨트롤러에서 addBook.jsp라는 페이지를 제공하는 코드이다. 이때 Post로 전해지는 파라미터들을 자동으로 매핑하고 싶다면 반드시 2개의 케이스중 하나를 선택해야 합니다.

- Case 1 : Book객체를 자동으로 생성하여 뷰에 전달하는 방식입니다.
- Case 2 : Book객체를 수동으로 생성하여 모델 객체에 담아서 전달합니다.



스텝 2:

뷰 생성하기

뷰 생성하기

- 파일위치 : /WEB-INF/views/addBook.jsp

BookMarket/src/com/springmvc/controller/BookController.java

```

<%@ page contentType="text/html; charset=utf-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<html>
<head>
<link href="<c:url value="/resources/css/bootstrap.min.css"/>" rel="stylesheet">
<title>도서 등록</title>
</head>
<body>
  <nav class="navbar navbar-expand navbar-dark bg-dark">
    <div class="container">
      <div class="navbar-header">
        <a class="navbar-brand" href="/home">Home</a>
      </div>
    </div>
  </nav>
  <div class="jumbotron">
    <div class="container">
      <h1 class="display-3">도서 등록</h1>
    </div>
  </div>
  <div class="container">
    <form:form modelAttribute = "NewBook" class="form-horizontal" >
      <fieldset>
        <legend>${addTitle}</legend>
        <div class="form-group row">
          <label class="col-sm-2 control-label">도서 ID</label>
          <div class="col-sm-3">
            <form:input path="bookId" class="form-control"/>
          </div>
        </div>
        <div class="form-group row">
          <label class="col-sm-2 control-label">도서명</label>
          <div class="col-sm-3">
            <form:input path="name" class="form-control"/>
          </div>
        </div>
        <div class="form-group row">
          <label class="col-sm-2 control-label">가격</label>
          <div class="col-sm-3">
            <form:input path="unitPrice" class="form-control"/>
          </div>
        </div>
      </fieldset>
    </form:form>
  </div>
</div>

```

```

<div class="form-group row">
    <label class="col-sm-2 control-label" >저자</label>
    <div class="col-sm-3">
        <form:input path="author" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >상세정보</label>
    <div class="col-sm-5">
        <form:textarea path="description" cols="50" rows="2" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >출판사</label>
    <div class="col-sm-3">
        <form:input path="publisher" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >분야</label>
    <div class="col-sm-3">
        <form:input path="category" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >재고수 </label>
    <div class="col-sm-3">
        <form:input path="unitsInStock" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >출판일</label>
    <div class="col-sm-3">
        <form:input path="releaseDate" class="form-control"/>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 control-label" >상태</label>
    <div class="col-sm-3">
        <form:radio button path="condition" value="New" /> New
        <form:radio button path="condition" value="Old" /> Old
        <form:radio button path="condition" value="E-Book" /> E-Book
    </div>
</div>
<div class="form-group row">
    <div class="col-sm-offset-2 col-sm-10" >
        <input type="submit" class="btn btn-primary" value ="등록"/>
    </div>
</div>
</fieldset>
</form:form>
<hr>
<footer> 이하 생략

```

스프링 폼을 사용하기 위한 구성요소

- 스프링 폼태그 라이브러리 설정 : 태그 라이브러리 추가
- <form:form> 태그의 속성에 전달받은 Book 객체를 명시
- <form:input> 태그에 연결할 변수의 이름을 path속성으로 명시



스텝 5 :

결과 확인하기



localhost:8080/BookMarket/books/add

Home

도서 등록

신규 도서 등록

도서 ID

도서명

가격

저자

상세정보

출판사

분야

재고수

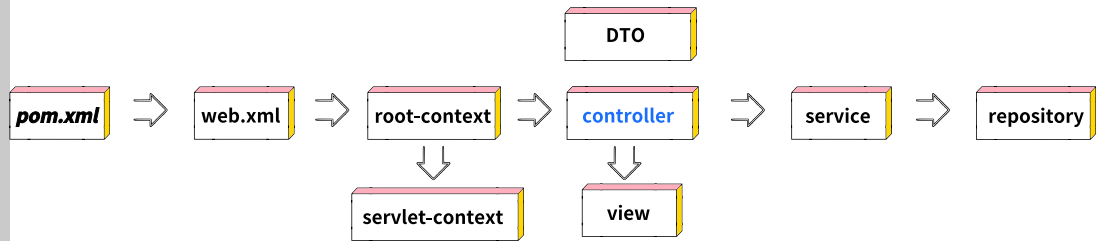
출판일

상태 ☐ New ☐ Old ☐ E-Book

등록

© BookMarket

9. 도서등록을 위한 데이터 바인딩: 전처리



스텝 1:

컨트롤러에서 파라미터 처리하기

클라이언트에서 폼페이지에 내용을 입력한 후 Submit 처리를 하면 Post방식으로 다음과 같은 요청이 발생한다.

"http://localhost:8080/BookMarket/books/add"

- 클래스 매핑 : /books
- 메서드 매핑 : /add "Post"
- 파라미터 : 없음

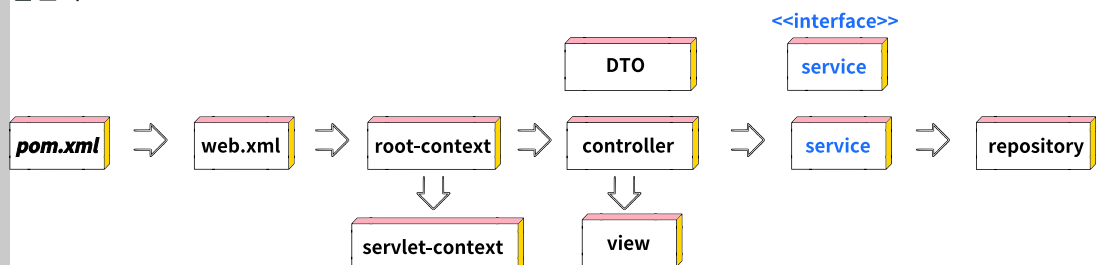
BookMarket/src/com/springmvc/controller/BookController.java

```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @PostMapping("/add")
    public String submitAddNewBook(@ModelAttribute("NewBook") Book book) {
        bookService.setNewBook(book);
        return "redirect:/books";
    }
}
  
```

폼페이지의 모든 파라미터는 DTO 객체의 변수에 모든 내용이 자동으로 book 매개변수에 매핑되어 대 입된다.



스텝 2:

서비스 클래스 수정하기

서비스 클래스 수정

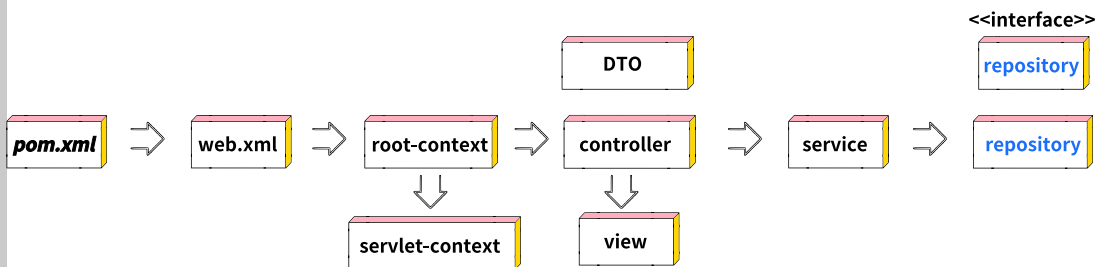
- 패키지 : com.springmvc.service
- 파일이름 : BookService.java

BookMarket/src/com/springmvc/service/BookService.java

```
public interface BookService {  
    void setNewBook(Book book);  
}
```

BookMarket/src/com/springmvc/service/BookServiceImpl.java

```
@Service  
public class BookServiceImpl implements BookService{  
    @Autowired  
    private BookRepository bookRepository;  
  
    public void setNewBook(Book book) {  
        bookRepository.setNewBook(book);  
    }  
}
```



스텝 4:

리포지토리 클래스 수정하기

리포지토리 클래스 수정

- 패키지 : com.springmvc.repository
- 파일이름 : BookRepository.java

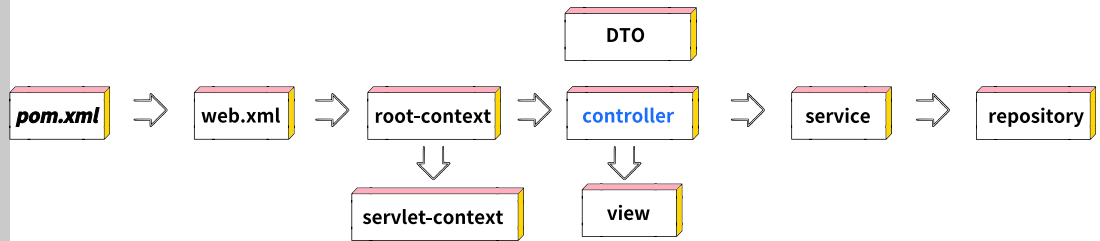
BookMarket/src/com/springmvc/repository/BookRepository.java

```
public interface BookRepository {  
    void setNewBook(Book book);  
}
```

BookMarket/src/com/springmvc/repository/BookRepositoryimpl.java

```
@Repository  
public class BookRepositoryImpl implements BookRepository{  
    private List<Book> listOfBooks = new ArrayList<Book>();  
  
    public void setNewBook(Book book) {  
        listOfBooks.add(book);  
    }  
}
```

10. 데이터 바인딩 필터링: 전처리



스텝1:

컨트롤러에서 파라미터를 선별적으로 처리하기

클라이언트에서 폼페이지에 내용을 입력한 후 Submit 처리를 하면 Post방식으로 다음과 같은 요청이 발생한다. 이후 선별적으로 DTO에 데이터를 바인딩 할 수 있다.

"http://localhost:8080/BookMarket/books/add"

- 클래스 매핑 : /books
- 메서드 매핑 : /add "Post"
- 파라미터 : 없음

BookMarket/src/com/springmvc/controller/BookController.java

```

@Controller
@RequestMapping("/books")
public class BookController {
    @Autowired
    private BookService bookService;

    @InitBinder
    public void initBinder(WebDataBinder binder) {
        binder.setAllowedFields("bookId", "name", "unitPrice", "author", "description",
            "publisher", "category", "unitsInStock", "totalPages", "releaseDate", "condition");
    }
}
  
```

위의 setAllowedFields() 메서드의 항목에 없으면 매핑이 허용되지 않음.